# Building a Tic-Tac-Toe Game With Minimax

Roger Yeoh

## 1 INTRODUCTION

Tic-tac-toe is an abstract strategy game that requires both sides to think deeply to win optimally. The game is based on a nine-grid pattern, “O” and “X” act as opponents, they take turns filling the nine gap with their symbol. The first one to get three symbols together in a horizontal, vertical or diagonal line win the game. It’s an excellent and suitable game to start with and delve into artificial intelligence algorithms. Because there exist a range of knowledge and intractable points that need to be focus on. Such as the turn system and the game status detection in the basic structure of game. Furthermore, the minimax algorithm includes the implementation of recursion, the programming of the decision tree, and the whole-process evaluation aimed at finding the best outcomes.

## 2 TWO-PLAYER TIC-TAC-TOE

To realize the project, it is necessary to begin with the fundamental two-player tic-tac-toe game. The program uses several fundamental and essential Python programming concepts, including lists, loops, conditionals, and functions. A 2D list was used to store the game state. Each gap is assigned a value of 'X', 'O', or a blank space. The check_winner() function checks if any row, any column or

any diagonal has the same symbol. The is_draw() function checks if all cells are filled without any player winning, by using a one-line expression with a nested loop to check all cells. In general, a large loop is constructed to repeatedly ask for input for each gap in turns, check input validity, place symbols, check the game status and show who's winning. In order to enhance the playability of the game, the tkinter module is imported to display the game in a graphical user interface. Furthermore, three useful buttons— Undo, Redo and Scoreboard are added below the game board.

## 3 IMPLEMENTATION OF MINIMAX

Subsequently, an attempt will be made to implement minimax into the fundamental game. But before proceeding further, it is imperative to review the underlying principle. Minimax is a decision rule that is used in artificial intelligence, decision theory, combinatorial game theory and statistics for minimizing the possible loss in a worst-case scenario. The minimax algorithm works by simulating all possible future outcomes. By using recursion, it builds a decision tree, where each node represents a possible game state. The algorithm assigns a score to each final state (e.g.: +1 for a win, -1 for a loss, and 0 for a draw). Then, it backtracks through the tree and makes the right choice that leads to the best possible outcome for the AI.

In the context of the game tic-tac-toe, the implementation of minimax could be divided into multiple sections. Firstly, the fundamental focus is on the basic minimax algorithms used in turn-based games, between two players — AI and human. Second, AI needs to construct a game tree while evaluating the possible future outcomes. Third, the goals of the algorithms are to choose the move with

the minimum score at the min nodes, and choosing the one with the maximum score with at max nodes. Lastly, the perfect strategy is considered throughout the whole module to ensure AI play optimally.

By adding a minimax module to the original code, the game is now available to play against AI in easy mode. It should be mention that easy mode would only adopt one level of evaluating, then the choices are made randomly.

```
def find_best_ai_move(self):
    # Step 1: Check if AI can win immediately
    for i in range(3):
        for j in range(3):
            if self.board[i][j] == "":
                self.board[i][j] = "O"
                if self.check_winner("O"):
                    self.board[i][j] = ""
                    return (i, j)  # if the place availble for winning is founded, return immediately
                self.board[i][j] = ""

    # Step 2: Block player win
    for i in range(3):
        for j in range(3):
            if self.board[i][j] == "":
                self.board[i][j] = "X"
                if self.check_winner("X"):
                    self.board[i][j] = ""
                    return (i, j)  # stop the player if it will win for the next step
                self.board[i][j] = ""

    # Step 3: Choose random move (for easy difficulty)
    available = [(i, j) for i in range(3) for j in range(3) if self.board[i][j] == ""]
    if available:
        return random.choice(available)  # choose a blank space randomly
    return None  # nowhere to go
```


The module is divided into three steps. Step 1 checks if AI can win immediately  in the next move. If the place is found, AI will fill its symbol in and win the game. Instead, if the place is not found, AI will try to block the player in their next move by step 2. In easy mode, AI only detects the current status to find out if the player will win at any places in the next step, and

fill the possible gap. Otherwise, if either player or AI can't win in their next step and there are still vacancies, AI will choose a random gap to fill in.

Subsequently, the minimax algorithm would get a boost in difficulty.

```
def best_move (self):
    best_score = float("-inf")   # Start with the lowest possible score
    move = None   # To store the best move coordinates
    for i in range (3):
        for j in range (3):
            if self.board[i][j] == '':  # Check if the cell is empty
                self.board[i][j] = "O"   # Try placing 'O' (AI) in this cell
                score = self.minimax(self.board, 0, False) # Evaluate this move using Minimax
                self.board[i][j] = ""   # Undo the move (backtrack)
                if score > best_score:   # If this move is better than previous best
                    best_score = score   # Update best score
                    move = (i, j)   # Save this move as the best one
return move   # Return the best move found

def minimax(self, board, depth, is_maximizing):
    if self.check_winner("O", board): #If AI wins in this state
        return 1 # Return highest score
    elif self. check_winner ("X", board): # If player wins
        return -1 # Return lowest score
    elif self.is_draw(board): # If board is full with no winner
        return 0 # Return neutral score

if is_maximizing: # AI's turn (maximize score)
    best_score = float("-inf")
    for i in range (3):
        for j in range (3):
            if board[i][i] == "":
                board[i][i] = "O" # Try placing 'O'
                score = self.minimax(board, depth + 1, False) # Recursively simulate player's response
                board[i][j]= "" # Undo move (backtrack)
                best_score = max(score, best_score) # Pick the best (max) outcome
    return best_score
else: # Player's turn (minimize score)
    best_score = float ("inf")
        for i in range (3):
            for j in range (3):
                if board[i][j] == "":
                    board[i][i] = "X" # Try placing 'X'
                    score = self.minimax(board, depth + 1, True) # Recursively simulate AI's response
                    board[i][j] = "" # Undo move (backtrack)
                    best_score = min(score, best_score) # Pick the worst (min) for AI
    return best_score
```

The minimax module is divided into two steps. The first step tries all possible moves by using the minimax algorithm to simulate the outcomes. The move that maximizes the chance of winning or at least ends in a draw, so it never plays a

suboptimal move. Furthermore, step 2 employs a recursive search algorithm, unsuitable move simulations would be undone through backtracking as a result. By following these methods, AI always chooses the option with the best result. Concurrently, all possible strategies of the player are anticipated and countered in advance.

After conducting a series of trials, the AI was proven to be unbeatable in this game. The reason for this, which differentiates unbeatable mode from easy mode is that the code conducts a range of new logics. It performs a complete game tree search, exploring and evaluating every possible move by both players through recursive decision-making until the end of the game. Score-based strategies are also used in this process. AI always selects the move that maximizes its chances of winning or, at worst, results in a draw, by backtracking. In addition, this AI does not skip any branches (no pruning) and avoids randomness. It guarantees the mathematically best move.

# 4 CONCLUSION

Artificial intelligence has successfully solved the game of tic-tac-toe with the minimax algorithm, which makes optimal choices at every turn, ensuring a win against the human player, or at worst, a draw. The AI chooses the most beneficial moves at each decision point through a complete game tree search and a score-based system, ensuring the optimal solution is achieved. On the other hand, although the algorithm is easy to run in the program of tic-tac-toe, the search is relatively inefficient due to the lack pruning. It is not applicable to more complex

and larger games as well as programs . In the process of constructing the game, I gained a deeper understanding of the core concepts of recursion, game trees and decision-making strategies, which provided me with further insight into AI.